

АВТОМАТИЗАЦИЯ ТЕСТИРОВАНИЯ

Автоматизация тестирования

Автоматизация тестирования — лучший способ повысить эффективность, покрытие тестами и скорость выполнения тестирования ПО.

Цель автоматизации — сократить количество тестовых случаев, которые нужно запустить вручную, а не полностью исключить ручное тестирование

Автоматизированное тестирование ПО важно потому, что:

- ✓ Ручное тестирование всех рабочих процессов, всех полей, всех негативных сценариев требует много времени и денег.
- ✓ Трудно вручную тестировать многоязычные сайты.
- ✓ Автоматизация тестирования в тестировании ПО не требует присмотра. Можно запустить автоматизированный тест без присмотра (в течение ночи).
- ✓ Автоматизация тестирования увеличивает скорость выполнения тестов.
- ✓ Автоматизация помогает увеличить тестовое покрытие.
- ✓ Ручное тестирование может стать скучным, подтверждено ошибкам из-за человеческого фактора.

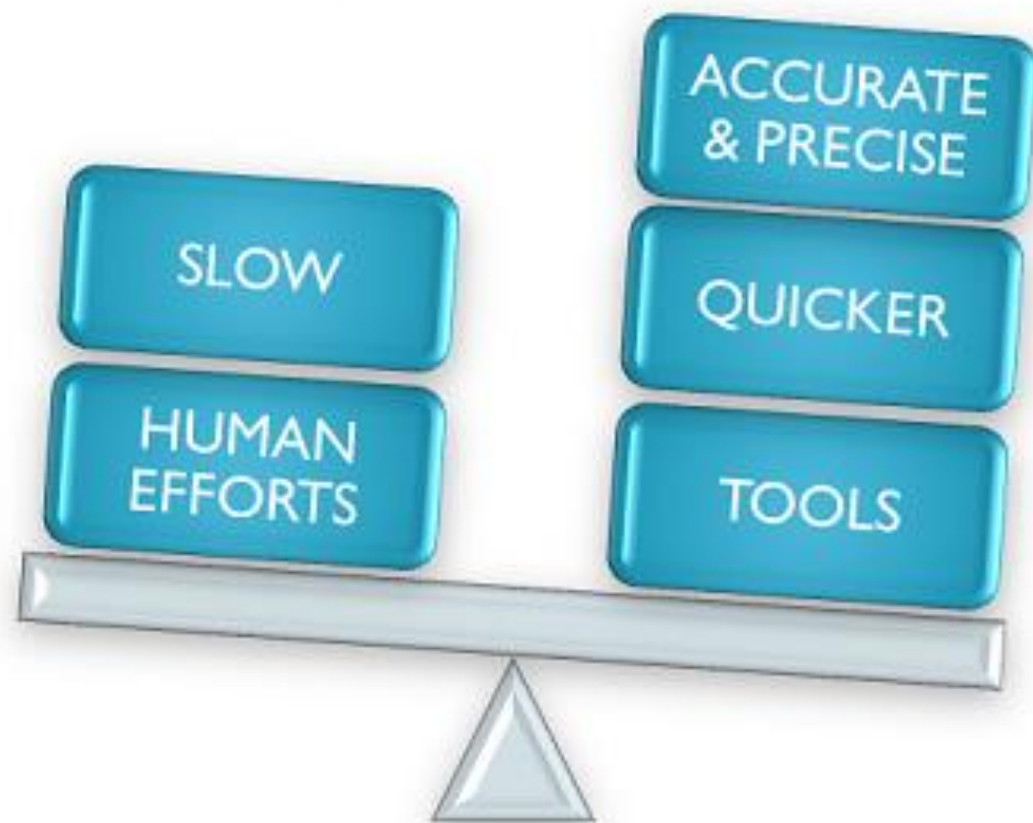
Преимущества автоматизации тестирования:

- На 70% быстрее ручного тестирования
- Более широкий охват тестирования функций приложения
- Надежные результаты
- Обеспечить последовательность
- Экономит время и деньги
- Повышает точность
- Вмешательство человека во время выполнения не требуется.
- Увеличивает эффективность
- Лучшая скорость выполнения тестов
- Тестовые сценарии, пригодные для повторного использования
- Тестирует часто и тщательно
- Раннее время выхода на рынок

Минусы автоматизированного тестирования:

- Без человеческого фактора **сложно получить представление о визуальных аспектах пользовательского интерфейса**, таких как цвета, шрифты, размеры, контрастность или размеры кнопок.
- Инструменты для проведения автоматизированного тестирования могут быть дорогими, что **может увеличить стоимость проекта тестирования**.
- Инструмент тестирования автоматизации пока не является полностью надежным. **Каждый инструмент автоматизации имеет свои ограничения**, что снижает область автоматизации.
- Отладка тестового скрипта — еще одна важная проблема в автоматизированном тестировании. **Поддержка тестов обходится дорого.**

MANUAL VS AUTOMATION



Параметр	Автоматизированное тестирование	Ручное тестирование
Время обработки	 Автоматизированное тестирование значительно быстрее ручного подхода.	 Ручное тестирование занимает много времени и требует человеческих ресурсов.
Исследовательское тестирование	 Автоматизация не допускает выборочного тестирования	 Исследовательское тестирование возможно в ручном тестировании
Первоначальные инвестиции	Первоначальные инвестиции в автоматизированное тестирование выше. Хотя окупаемость инвестиций в долгосрочной перспективе выше.	Первоначальные инвестиции в ручное тестирование сравнительно ниже. Окупаемость инвестиций ниже по сравнению с автоматизированным тестированием в долгосрочной перспективе.

Параметр	Автоматизированное тестирование	Ручное тестирование
Изменение пользовательского интерфейса	 Даже для незначительного изменения пользовательского интерфейса АУТ необходимо изменить сценарии автоматизированного тестирования, чтобы они работали так, как ожидается.	 Небольшие изменения, такие как изменение идентификатора, класса и т. д. кнопки, не мешают выполнению ручного тестирования.
Инвестиции	Требуются инвестиции в инструменты тестирования, а также в инженеров по автоматизации.	Необходимы инвестиции в человеческие ресурсы.
Экономически эффективно	Невыгодно для регрессии малого объема	Невыгодно для регрессии большого объема.

Параметр	Автоматизированное тестирование	Ручное тестирование
Тестирование производительности	+ Тесты производительности, такие как нагрузочное тестирование, стресс-тестирование, пиковое тестирование и т. д., должны в обязательном порядке тестироваться с помощью инструмента автоматизации.	■ Тестирование производительности невозможно выполнить вручную.
Параллельное выполнение	+ Это тестирование можно выполнять на разных операционных платформах параллельно, что сокращает время выполнения теста.	■ Ручные тесты можно выполнять параллельно, но для этого потребуется увеличить человеческие ресурсы, что является дорогостоящим.

Параметр	Автоматизированное тестирование	Ручное тестирование
Знание программирования	 Знание программирования является обязательным условием при автоматизированном тестировании.	 Нет необходимости в программировании при ручном тестировании.
Документация	Автоматизированные тесты действуют как документ, предоставляющий обучающую ценность, особенно для автоматизированных тестовых случаев. Новый разработчик может изучить тестовые случаи и быстро понять кодовую базу.	Ручные тестовые случаи не представляют никакой обучающей ценности

Какие тестовые случаи автоматизировать?

Тестовые случаи для автоматизации можно выбрать, используя следующий критерий для повышения рентабельности инвестиций в автоматизацию:

- ✓ **Критически важные для бизнеса тестовые случаи.**
- ✓ **Тестовые случаи, которые выполняются многократно.**
- ✓ **Тестовые случаи, которые очень утомительны или сложны для выполнения вручную.**
- ✓ **Тестовые случаи, требующие много времени.**

Для автоматизации не подходят:

- ✓ Тестовые случаи, которые были разработаны заново и не были выполнены вручную хотя бы один раз.
- ✓ Тестовые случаи, требования к которым часто меняются.
- ✓ Специфические тестовые случаи, которые выполняются редко или один раз.
- ✓ Визуальные/субъективные проверки (если нет специализированных инструментов)

Как оценить целесообразность и готовности проекта к внедрению автоматизированного тестирования

✓ Чек-лист «Готов ли проект к автоматизации?»

1. Стратегия и цели

- ✓ Чётко определены бизнес-цели автоматизации (сокращение времени, повышение качества, ускорение релизов)
- ✓ Выделены ресурсы: время, бюджет, специалисты для поддержки автотестов
- ✓ Определены метрики успеха (например: снижение количества багов в продакшене, время прогона регресса)

2. Стабильность продукта

- ✓ Основные функции и пользовательские сценарии уже определены и стабильны
- ✓ Интерфейс (UI) или API не меняются кардинально каждую итерацию
- ✓ Есть чёткая документация на функционал или пользовательские истории

- ✓ Чек-лист «Готов ли проект к автоматизации?»

3. Техническая готовность

- ✓ Приложение имеет тестируемую архитектуру (разделение слоёв, внедрение зависимостей)
- ✓ Есть возможность запускать приложение в изолированном тестовом окружении
- ✓ Есть доступ к логам, базам данных для интеграционного тестирования

4. Инфраструктура и процессы

- ✓ Используется система контроля версий (Git)
- ✓ Настроен CI/CD или есть возможность его внедрить
- ✓ Есть выделенные тестовые окружения (dev, staging, test)

5. Компетенции команды

6. Приоритизация: что автоматизировать в первую очередь?

Тестовое окружение / Тестовая среда / Тестовый стенд

В терминах ISTQB Glossary v4.0 (2023), тестовая среда определяется как «аппаратные, программные средства и конфигурации, в которых будет выполняться тестирование» (раздел *Test Environment*).

Без этой среды невозможны проверки кода, интеграции, производительности и безопасности. Она может быть простой — один сервер и база, а может включать десятки микросервисов, очереди сообщений и внешние интерфейсы.

Можно выделить среды: *Dev, Test, Integration, Staging, Performance, Production*

У каждой компании — свои правила. Где-то *Staging* используют как финальную точку перед продом, где-то *Staging* и *Prod* — одно и то же. В других случаях *Integration* может быть лишь логической меткой внутри CI-процесса. Но если попытаться усреднить и описать наиболее часто встречающиеся подходы — получится следующая таблица:

<https://habr.com/ru/articles/926122/>

таблица с кратким описанием основных видов тестовых (и не только) сред:

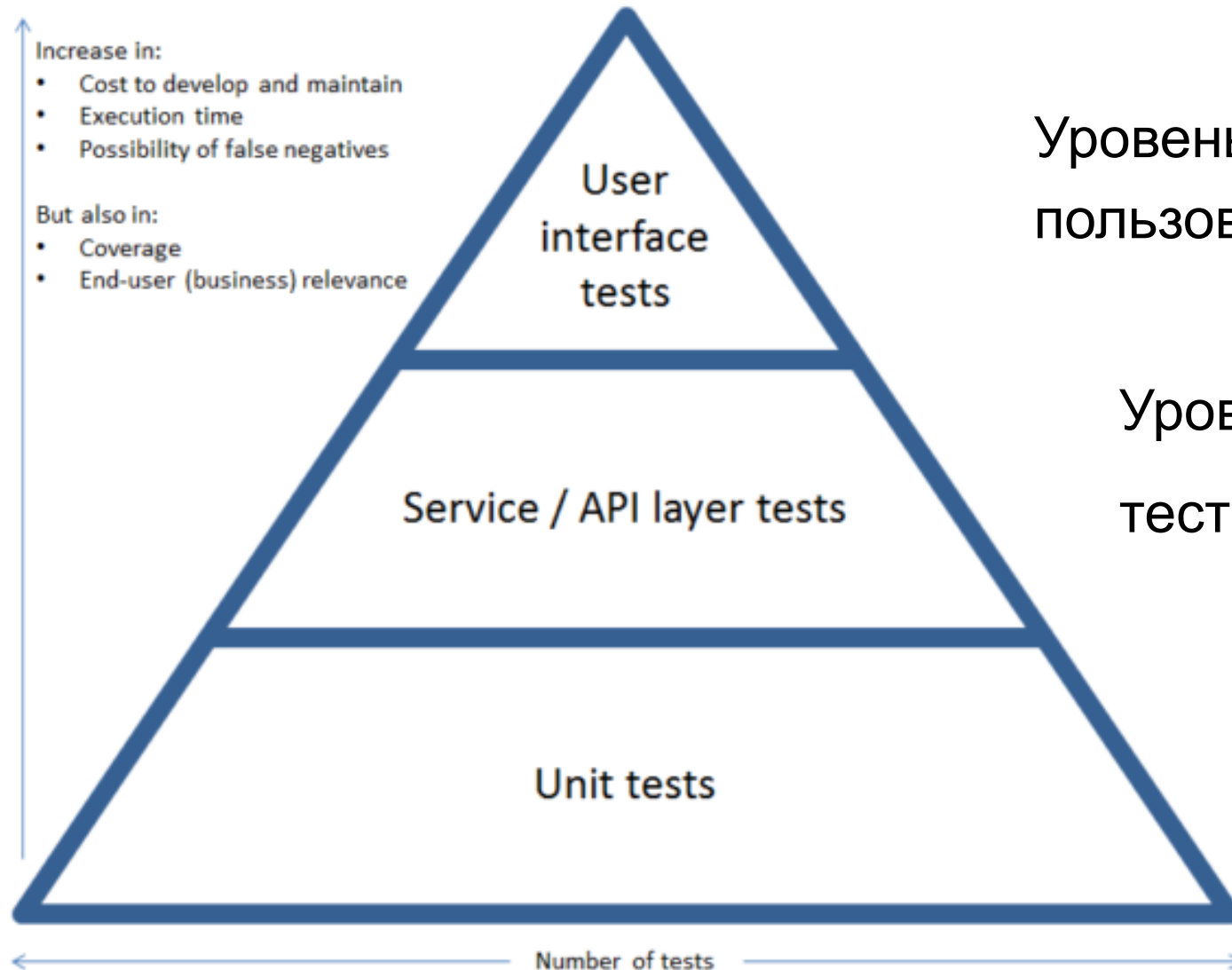
Название среды	Назначение	Кто использует	Особенности
Dev	Локальная разработка и отладка	Разработчики	Часто нестабильна, функции на ранней стадии разработки
Test	Ручное и автоматизированное тестирование	QA-инженеры	Часто очищается, может содержать заглушки и моки, могут быть параллельны под разные фичи или типы тестов.
Integration	Проверка взаимодействия компонентов	Разработчики, QA	Может быть несколько сервисов с разными ветками
Staging	Имитирует продакшн для финальных тестов	DevOps, QA, бизнес	Близка к продакшн, настоящие данные или их копия
Performance	Среда, идентичная продакшн	Performance QA, безопасность	Используется для нагрузочного тестирования: реалистичные объёмы данных, конфигурации продакшн-серверов, копии баз.
Production	Боевая среда	Пользователи	Только стабильный код. Тестирование минимальное, <i>A/B-тесты, канареечные релизы, мониторинг логов и метрик</i>

Фреймворк для автоматизации

Фреймворк — это набор руководств по автоматизации, которые помогают в:

- ✓ Поддержание последовательности тестирования
- ✓ Улучшает структурирование теста
- ✓ Минимальное использование кода
- ✓ Меньше обслуживания кода
- ✓ Улучшение возможности повторного использования
- ✓ Нетехнические тестировщики могут быть вовлечены в код
- ✓ Период обучения использованию инструмента может быть сокращен
- ✓ Включает данные везде, где это уместно

УРОВНИ АВТО-ТЕСТИРОВАНИЯ: Пирамида тестирования



Уровень тестирования через
пользовательский интерфейс (UI tests)

Уровень функционального
тестирование (non-UI tests)

Уровень модульного
тестирования (unit tests)

МОДУЛЬНОЕ ТЕСТИРОВАНИЕ

Процесс в программировании, позволяющий проверить на корректность отдельные модули исходного кода программы (Unit –тесты).

Идея состоит в том, чтобы писать тесты для каждой нетривиальной функции или метода.

Это позволяет достаточно быстро проверить, не привело ли очередное изменение кода к регрессии, то есть к появлению ошибок в уже оттестированных местах программы, а также облегчает обнаружение и устранение таких ошибок.

Пример юнит теста для метода Sum класса Calculator

```
class CalculatorTests
{
    public void Sum_2Plus5_7Returned()
    {
        // arrange
        var calc = new Calculator();
        // act
        var res = calc.Sum(2,5);
        // assert
        Assert.AreEqual(7, res);
    }
}
```

ФРЕЙМВОРКИ ДЛЯ UNIT-ТЕСТОВ

JUnit

×Unit.net

PHPUnit

pytest

MiniTest
Ruby

PHPUnit

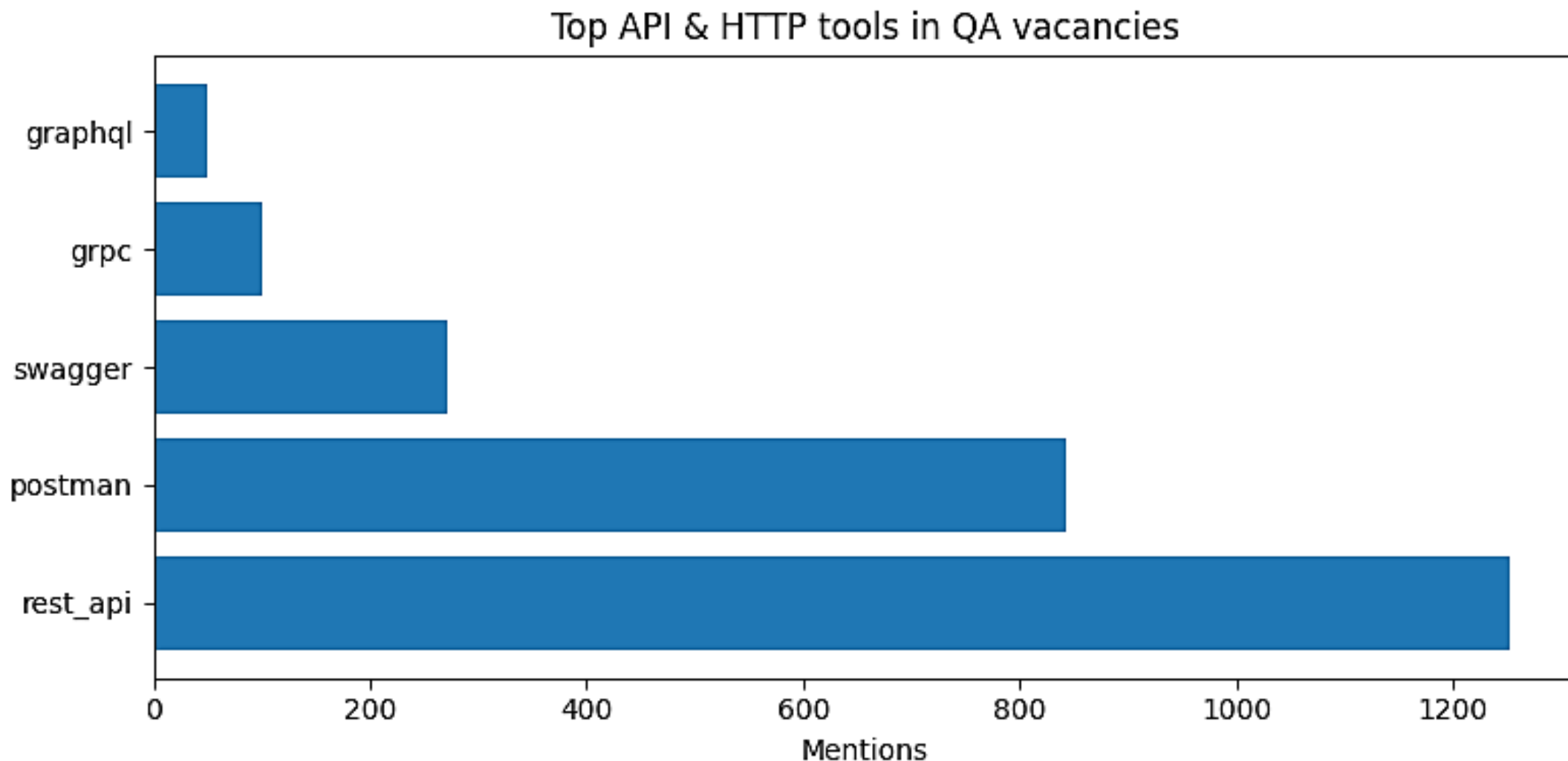
NON-UI ТЕСТИРОВАНИЕ

Процесс в программировании, позволяющий проверить работоспособность приложения используя программный интерфейс приложения (application programming interface, API).

Не всегда всю бизнес логику приложения можно протестировать через GUI слой. Это может быть особенностью реализации, которая прячет бизнес логику от пользователей. Именно по этой причине для команды тестирования может быть реализован доступ напрямую к функциональному слою, дающий возможность тестировать непосредственно бизнес логику приложения, минуя пользовательский интерфейс.

На графике показано распределение упоминаний инструментов и технологий, связанных с API и HTTP-взаимодействием, в QA-вакансиях за 2025 год.

<https://habr.com/ru/articles/982052/>



```
package CountriesRestTests;

...
public class GetTest {

    @Test
    public void getRequestFindCapital() throws JSONException {

        // выполняем запрос get для доступа ко всем параметрам ответа
        Response resp = get("http://restcountries.eu/rest/v1/name/belarus");

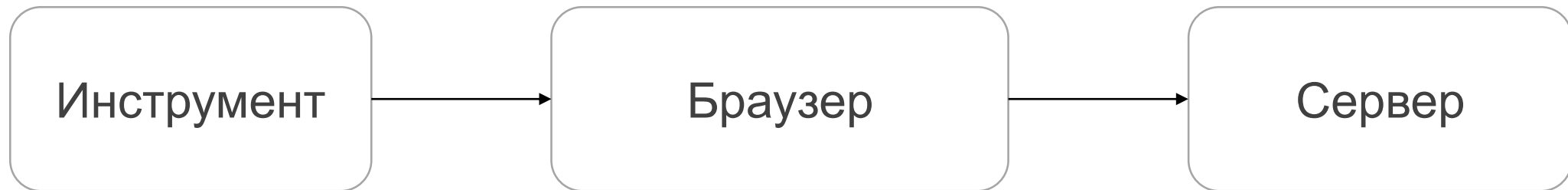
        JSONArray jsonResponse = new JSONArray(resp.asString());

        // получение параметра capital (столицы Беларуси)
        String capital = jsonResponse.getJSONObject(0).getString("capital");

        // проверка, что столицей является Минск
        AssertJUnit.assertEquals(capital, "Minsk");
    }
}
```


UI ТЕСТИРОВАНИЕ

Процесс в программировании, позволяющий проверить работоспособность и внешний вид приложения используя графический интерфейс приложения.



Популярные инструменты UI-тестирования в 2026 году:

- ✓ **Selenium WebDriver:** Остается самым популярным и универсальным решением с поддержкой многих языков (Java, Python, C#, JS).
- ✓ **Playwright:** Лидер по скорости и современным возможностям, обеспечивает высокую надежность E2E-тестов для современных веб-приложений.
- ✓ **Cypress:** JS-фреймворк, ориентированный на удобство разработчиков (Developer Experience) и быструю разработку тестов.
- ✓ **Selenide:** Лаконичная обертка над Selenium WebDriver для Java, упрощающая написание кода.
- ✓ **Appium:** Основной инструмент, если UI-тестирование веб-приложений требует поддержки мобильных браузеров (Android/iOS).

SELENIUM

Пример кода

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.htmlunit.HtmlUnitDriver;
public class Example {
    public static void main(String[] args) {
        WebDriver driver = new HtmlUnitDriver();
        driver.get("http://www.google.com");
        WebElement element = driver.findElement(By.name("q"));
        element.sendKeys("Lecture for automated testing!");
        element.submit();
        System.out.println("Page title is: " + driver.getTitle());
        driver.quit(); } }
```

ПРОЦЕСС АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ

- ✓ Подготовка и планирование;
- ✓ Дизайн и разработка;
- ✓ Прогон тестов и сопровождение.

ПРОЦЕСС АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ

- ✓ Подготовка и планирование;

- ✓ Дизайн и разр

- ✓ Прогон тестов

- ✓ Анализ требований клиента к автоматизации;
- ✓ Анализ тест-кейсов для ручного тестирования;
- ✓ Оценка пригодности приложения к автоматизации;
- ✓ Выбор инструмента автоматизации;
- ✓ Создание пилотных скриптов;
- ✓ Установка тестового окружения.

ПРОЦЕСС АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ

✓ Подготовка и планирование;

✓ **Дизайн и разработка;**

✓ Прогон тестов

- ✓ Дизайн архитектуры (модули, фреймворки);
- ✓ Подготовка тестовых данных;
- ✓ Определение повторно используемого кода и реализация многократно используемых функций, библиотек и фреймворков;
- ✓ Разработка и отладка тестовых скриптов.

ПРОЦЕСС АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ

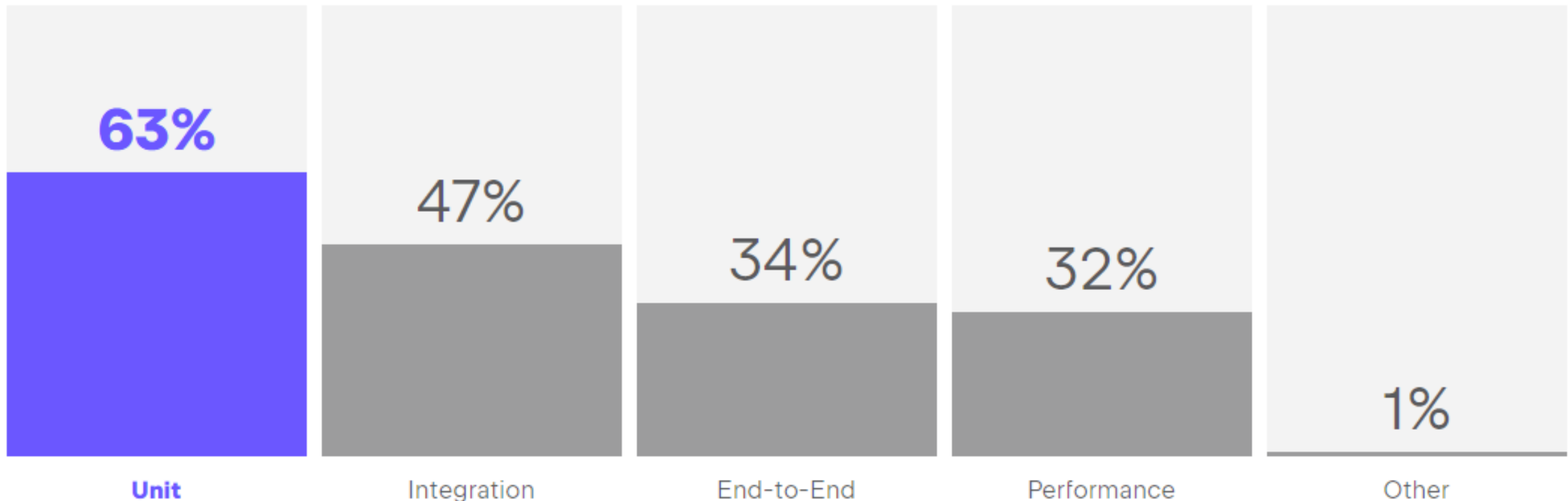
- ✓ Подготовка и планирование;
- ✓ Дизайн и разработка;
- ✓ **Прогон тестов и сопровождение.**

- ✓ Запуск автоматических скриптов
- ✓ Анализ результатов запуска
- ✓ Создание отчетов о результатах запуска
- ✓ Поддержание тестов в актуальном состоянии.

Исследования JetBrains 2023

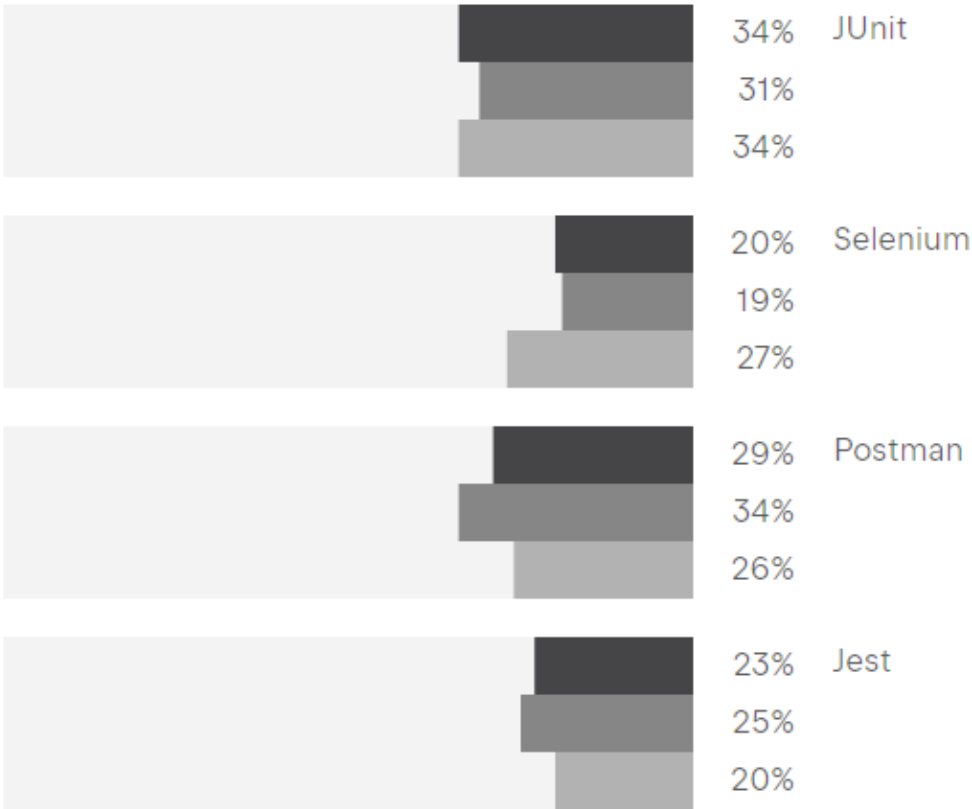
<https://www.jetbrains.com/lp/devecosystem-2023/testing/>

What types of tests do you have in your projects?

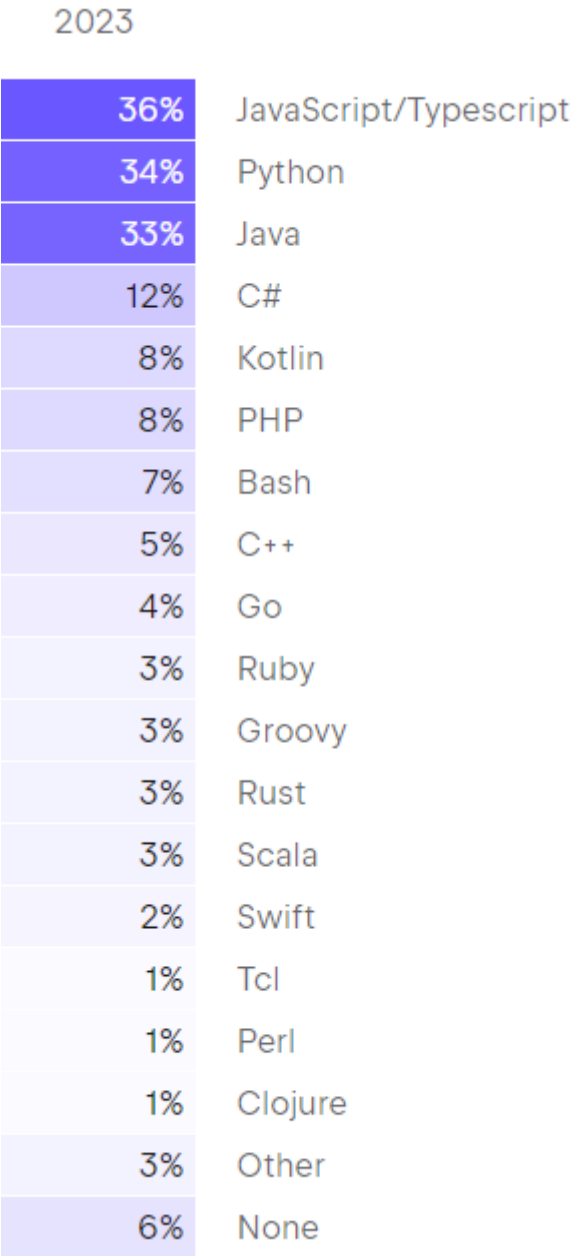


Which test frameworks, tools, and technologies do you use, if any?

2021
2022
2023



Which programming languages do you use for test automation in your project?



Что говорят специалисты?



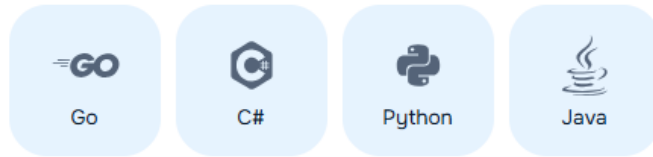
https://www.cnews.ru/news/line/2024-03-05_issledovanie_perfomans

Дмитрий Колесник, руководитель группы разработки инструментов тестирования и технического комитета по QA в Ozon Tech сказал: «Мы в целом не делим тестировщиков на ручных и автоматизаторов. Каждый инженер должен уметь и тестировать вручную, и писать код, чтобы покрыть автоматизацией свои тесты.

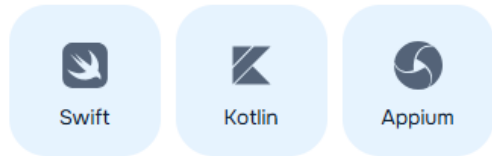
Говоря об универсальных разработках, мы делаем инструменты для подготовки тестовых данных. Например, наш сервис генерации тестовых данных позволяет создавать около 300 тыс. сущностей в день — использовать их можно как для ручных, так и для автотестов.

В Ozon постоянно обновляются требования и проходит примерно 4,5 тыс. релизов в неделю, поэтому стараемся автоматизировать все, что можем. Конечно, это не означает, что у нас вообще нет ручного тестирования — вручную проводим acceptance, тестирование новых функциональностей, зачастую проверяем мобильные приложения. Однако 90% всех кейсов автоматизированы».

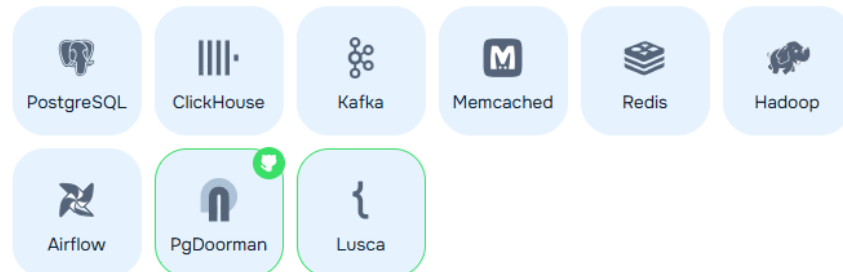
Backend



Mobile



Data



Infrastructure



ML/DS

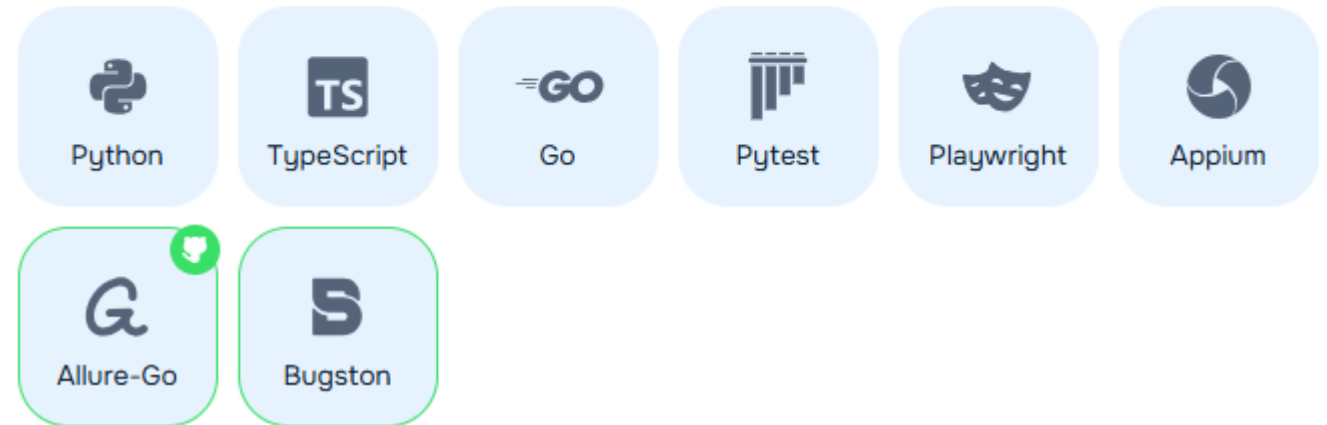


Под капотом у OZON

Frontend



QA



https://ozon.tech/?abt_att=1&origin_referer=www.google.com



QA лид на одном из крупных проектов Сбертех:

У нас тысячи тестов, их количество растёт. Без автоматизации мы будем постоянно отсекать и отсекать менее приоритетные кейсы, чтобы уложиться по времени, пока в какой то момент не останутся лишь самые критичные. К какому качеству продукта это приведёт, даже представить страшно.

Кстати, на текущем проекте примерно 15% от общего числа тестов у нас нельзя автоматизировать, и в среднем так выходит на каждом проекте.

Пирамида тестирования

*помогает понять как выстраивать
тесты на проекте*

Стандартная тестовая пирамида

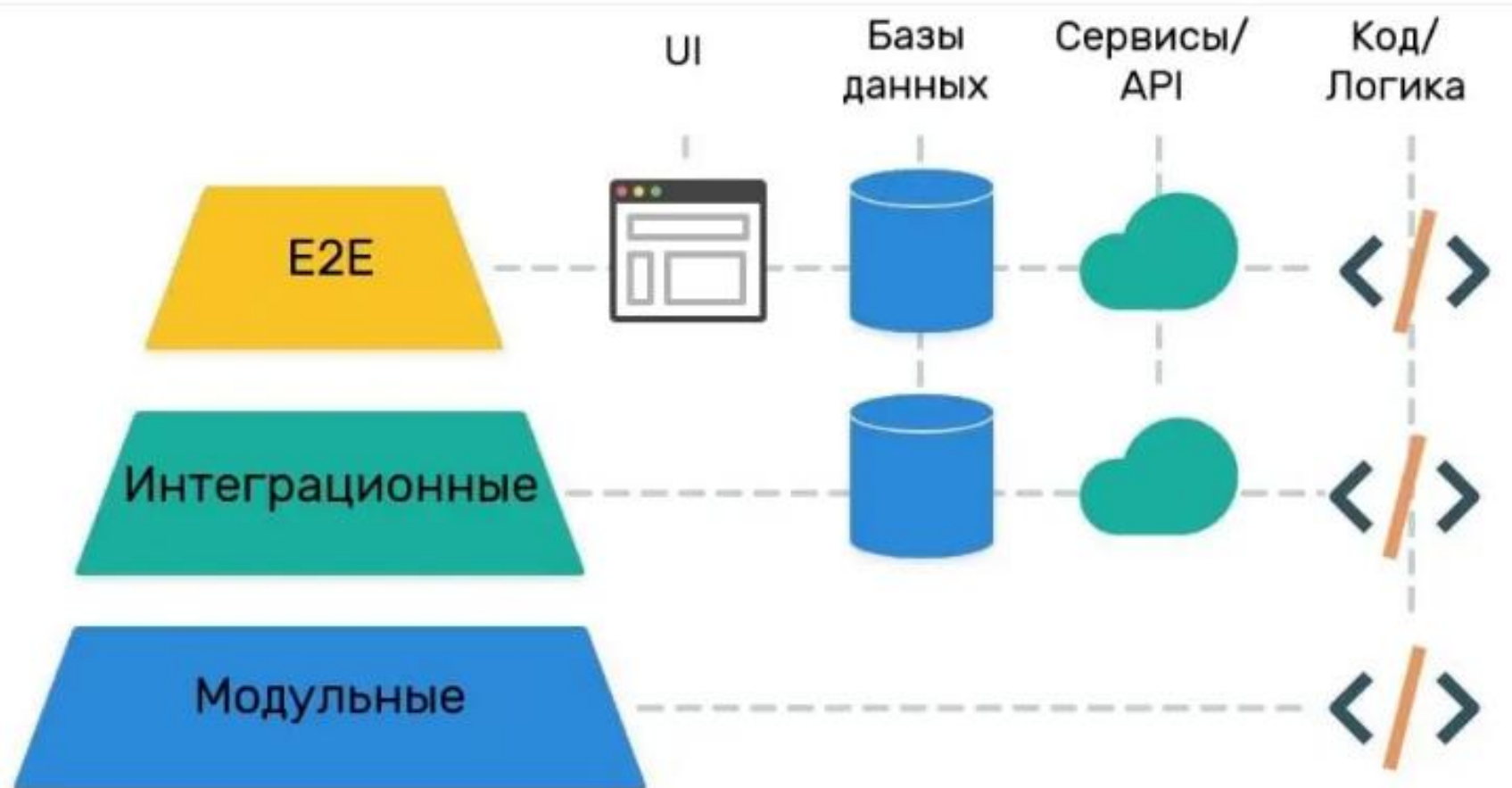


Тесты верхнего уровня (E2E), проверяют соблюдение требований к интерфейсу и общих требований к продукту. Из-за сложности и трудоемкости их количество по возможности стараются ограничить

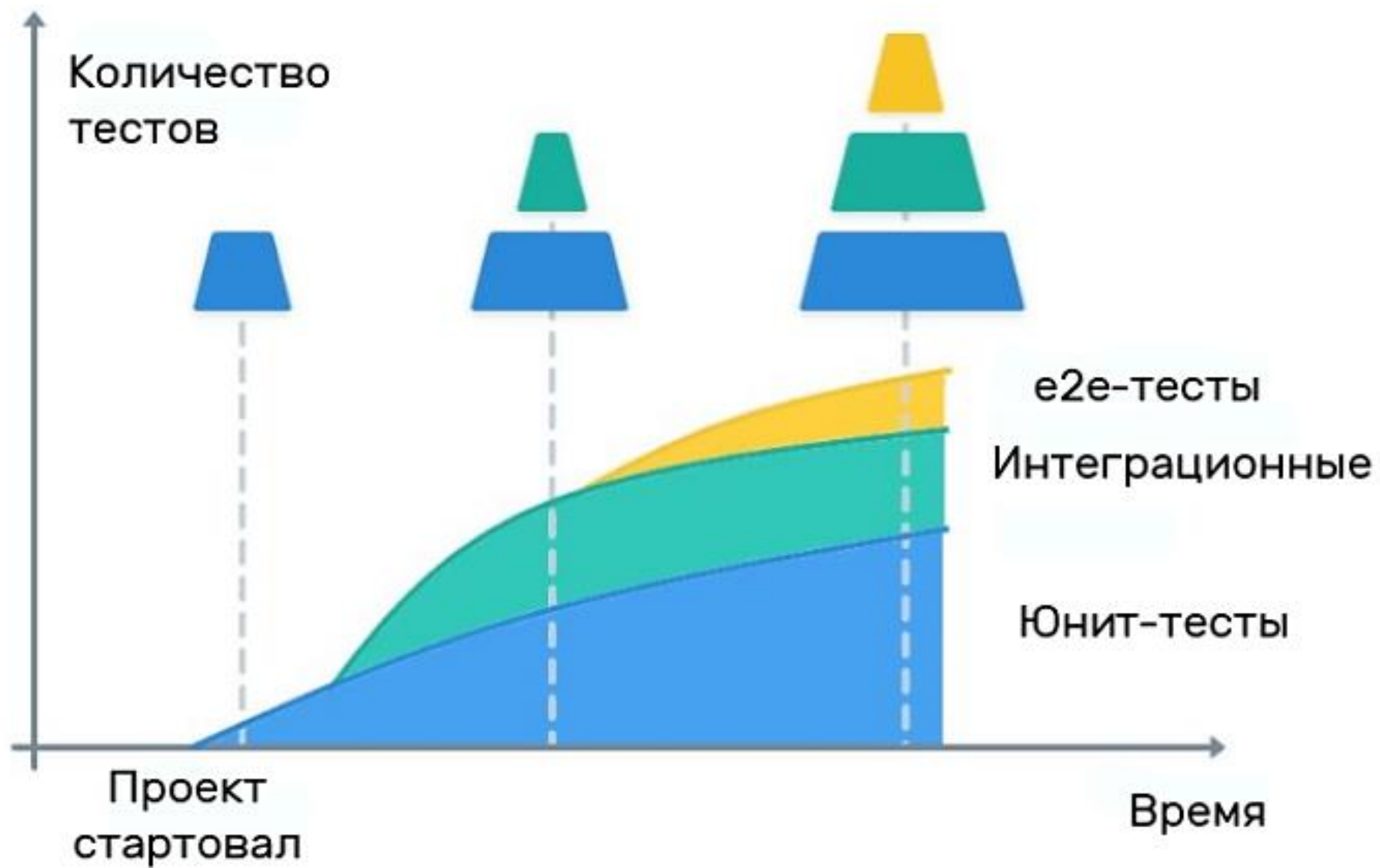
Имеются в виду интеграционные тесты, проверяющие взаимодействия между юнитами и более крупными чем юнит сущностями — компонентами.

Юнит-тестов больше чем всех остальных [вместе взятых], потому что в приложении как правило много модулей, соответственно модульных тестов требуется много, и они простые.

Самая старая, и все еще самая широко применяемая, как бы эталонная тестовая стратегия, впервые подробно описанная в книге Майкла Кона *“Succeeding with Agile”*.



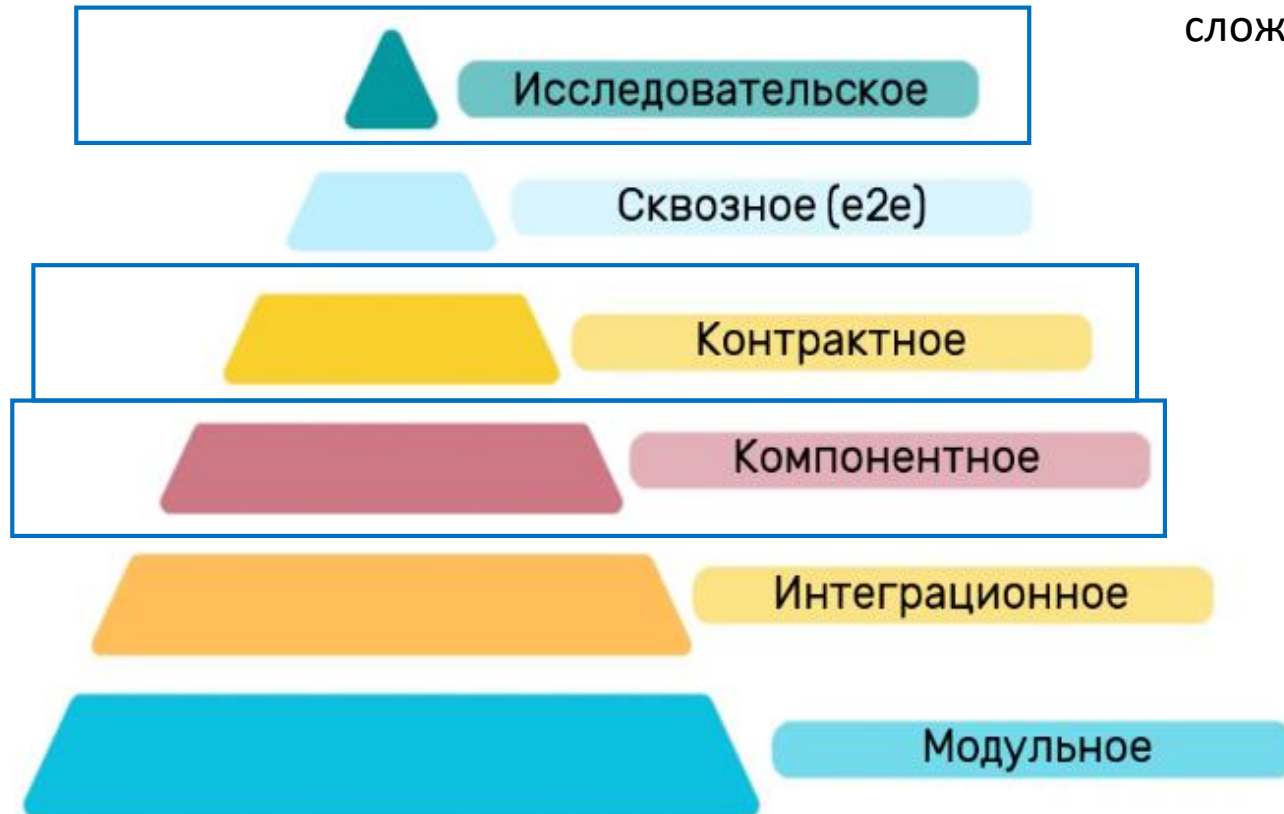
Тип тестов	Скорость выполнения
Юнит-тесты	0,01-0,1 секунды
Сервисные (интеграционные) тесты	1 секунда
Сквозные (e2e) и UI-тесты	10 секунд



Расширенная пирамида тестирования

Со временем стандартную пирамиду начали расширять добавлением отдельных **интеграционных контрактных тестов**, и **компонентных тестов**. Также в расширенной пирамиде появляются **ручные исследовательские тесты**.

При этом сохраняется закономерность «чем выше, тем меньше тестов»; чем ближе к концу цикла, тем сложнее тесты и меньше их количество.



Контрактное тестирование - это методология проверки, что две подсистемы (чаще всего два **микросервиса**) совместимы и могут беспрепятственно коммуницировать на основании контракта. Оно гарантирует, что изменения в одном сервисе не сломают другой, обеспечивая надежность микросервисной архитектуры.

Компонентное тестирование — это тестирование компонентов более крупных чем модули.

Песочные часы (DevOps)

В DevOps делают упор на тщательный мониторинг процессов. DevOps рассчитан на разработку с частыми деплоями. Поэтому к стандартной тестовой пирамиде «присоединяются» последующие процессы **Logging-Monitoring-Alerting**.

- **Журналирование (Logging)** — налаженное протоколирование (фиксация) информации о происходящем в приложении
- **Мониторинг (Monitoring)** — автоматизированное наблюдение за состоянием приложения и его серверной части
- **Оповещения (Alerting)** — налаженный процесс автоматических оповещений о проблемных точках приложения, чтобы их по возможности немедленно устранить

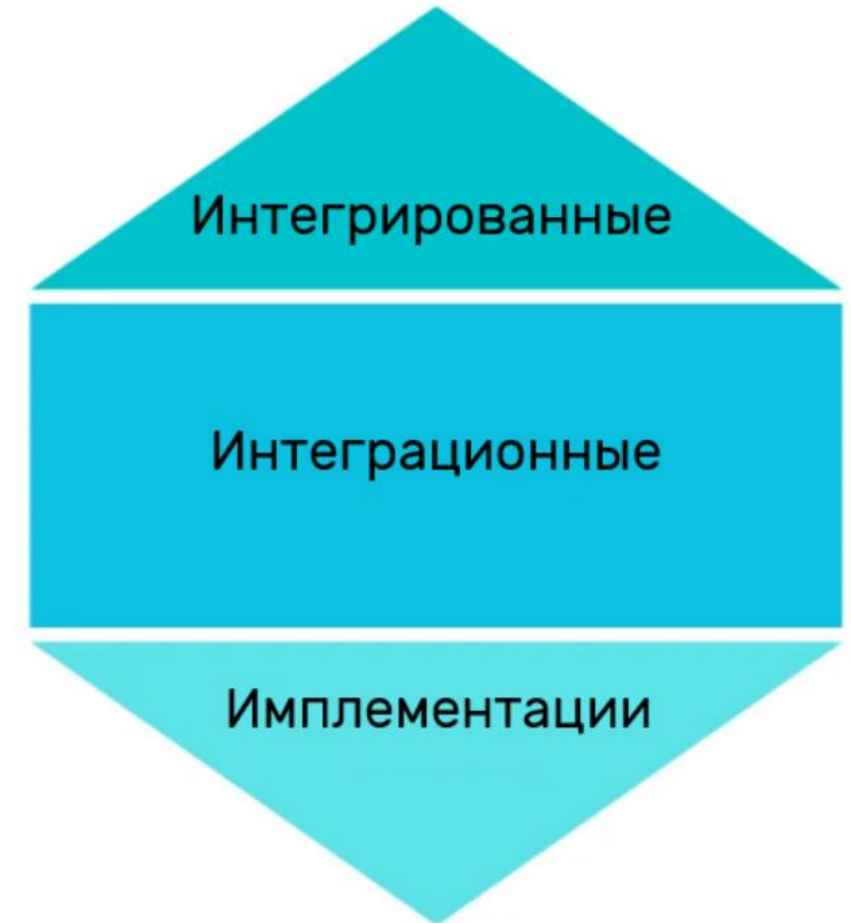


Сота

Специфическую тестовую пирамиду представили в компании **Spotify** в 2018г. Такая стратегия заточена под микросервисную архитектуру, в которой самая большая сложность часто состоит не как в самом сервисе, а как в его взаимодействиях с другими сервисами.

Сота, состоит из 3 слоев:

- **Интегрированные** (integrated) тесты — тесты, которые *проходят* или *падают* в зависимости от правильности взаимодействий с другой (внешней) системой.
- **Интеграционные** (integration) тесты проверяют правильность работы основного сервиса в более изолированном виде чем предыдущие, фокусируясь на точках взаимодействия и совершенствуя их работу.
- **Тесты имплементации** сфокусированы на изолированных частях кода, как бы специфический аналог юнит-тестирования.

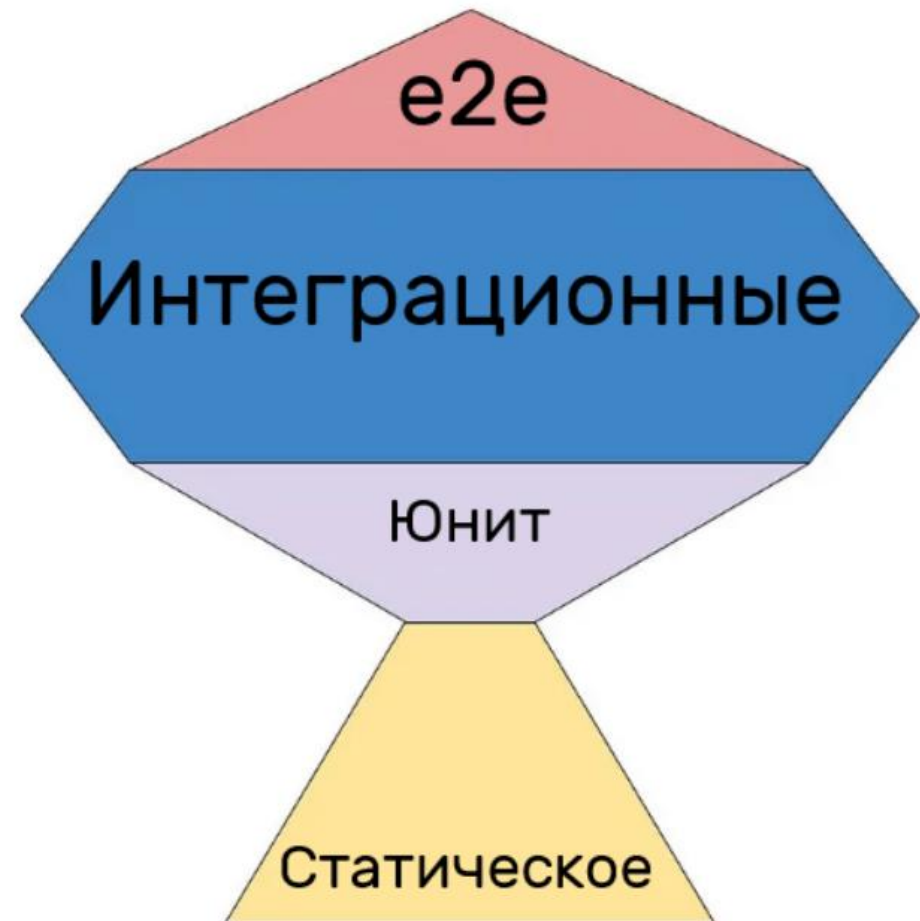


Кубок

Концепция представлена в 2018 году, признанным экспертом в QA Кеннетом Доддсом.

Автор концепции, исходя из своего опыта, полагает, что по мере продвижения проекта к верхушке тестовой пирамиды, уверенность в качестве тестов (а значит и количество их) должна быть максимальной на среднем уровне пирамиды — на ее интеграционном уровне.

Но также большое внимание должно уделяться предварительному уровню пирамиды — статическому тестированию, которое проводится перед модульным.



Вулкан

Еще один вариант тестовой пирамиды — «извергающийся вулкан». Это часто встречающийся вариант уровней тестирования в современной разработке.

Количество исследовательских и ручных тестов может быть достаточно большим, а вообще неопределенно и непредсказуемо, как и длительность ручного тестирования.

Поклонники стратегии «вулкана» полагают, что классическая пирамида уровней тестирования уже отжила свое, плохо учитывает рыночные риски проектов, что нужно больше опираться на риск-тестирование, и что старое-доброе ручное тестирование (включая исследовательское), и в отдаленном будущем нельзя заменить никогда и ничем.



Перевернутая пирамида тестирования

Часто встречается на практике.

Даже сейчас, с повсеместным внедрением Agile/Scrum/DevOps и прочих полезных методологий, разделение труда между Dev- и QA-отделами все еще очень отчетливое, **тестировщики не всегда имеют доступ к кодовой базе, и если разработчиков не заставляют писать юнит-тесты, то «фундаментные тесты», будь то юнит-, компонентные или интеграционные, просто некому писать**, и все возможное с QA продукта делают тестировщики. Вследствие чего количество «тестов верхнего уровня» закономерно увеличивается.

Также в современных проектах заказчики склонны требовать «релиз к сроку любой ценой», не особо вникая в перипетии взаимоотношений между отделами. Команды вынуждены регулярно демонстрировать заказчикам «то что есть», и быстрые e2e-тесты от QA-отдела — единственный разумный выход.

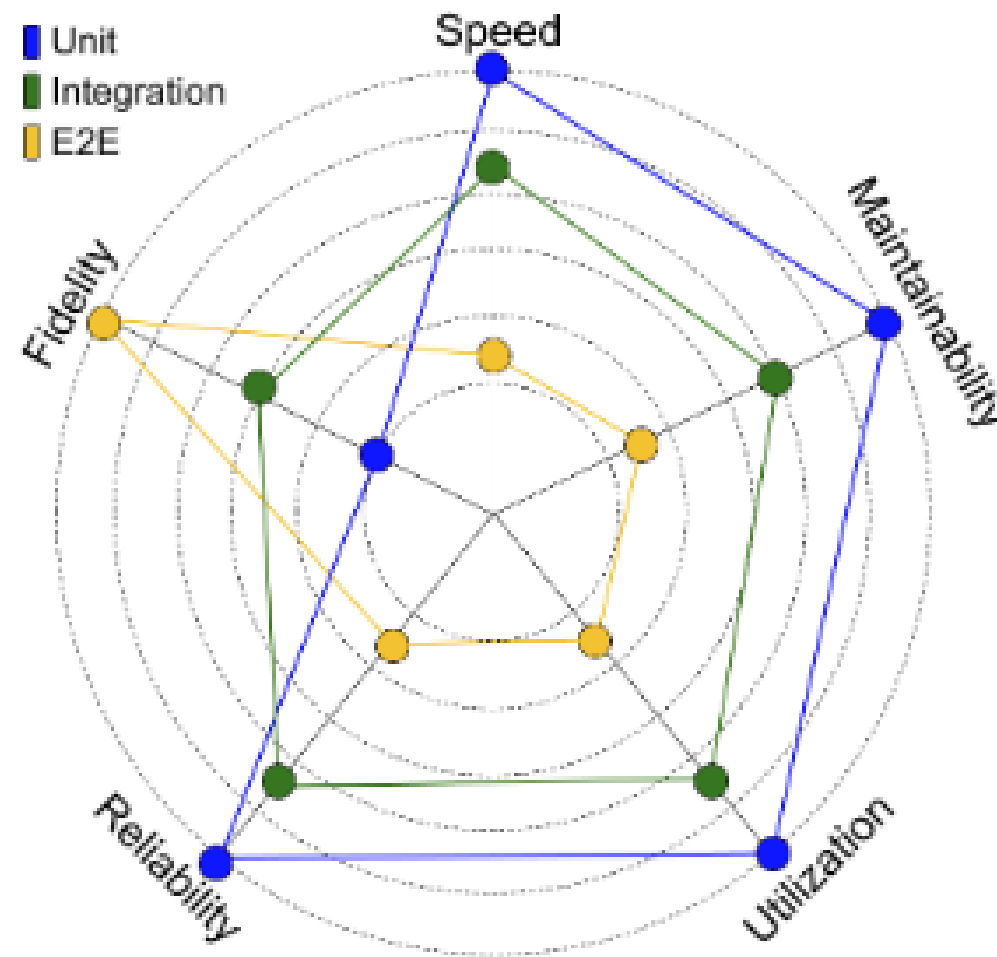


Мнемоника SMURF — простой способ запомнить 5 характеристик, которые необходимо учитывать при балансировке растущего тестового набора:

Скорость (**Speed**): Юнит-тесты намного быстрее других и могут выполняться чаще.

Удобство обслуживания (**Maintainability**): Совокупные затраты на отладку и поддержку тестов (всех типов) быстро растут. В большой системе больше кода, а это значит, что система больше подвержена изменению зависимостей и дрейфу требований, что приводит к увеличению объема работ по обслуживанию.

Утилизация ресурсов (**Utilization**): Тесты, использующие меньше ресурсов (память, диск, процессор), обходятся дешевле. Хороший тестовый набор оптимизирует использование ресурсов таким образом, чтобы оно не росло сверхлинейно с увеличением количества тестов. Юнит-тесты обычно обладают отличными характеристиками утилизации, потому что в них используются тестовые дублиры, и проверяются маленькие части системы.



Надежность (**Reliability**): Надежные тесты падают только при обнаружении реальной проблемы. Разбор нестабильных тестов в поисках проблем требует времени и ресурсов на повторное выполнение тестов. По мере роста системы и количества тестов в ней растет неопределенность (следовательно и нестабильность тестов), и ваш набор теряет надежность.

Точность (**Fidelity**): Точные тесты соответствуют реальным условиям работы системы (например, реальной нагрузке на БД и реальному трафику) и лучше предсказывают поведение на продакшене. **Интеграционные и сквозные тесты лучше отражают реальные условия, в то время как юнит-тесты вынуждены моделировать свое окружение, что приводит к расхождению между ожиданиями от тестов и реальностью.**

